

DEVELOPER

Developer Guide

Table of Contents

- 1. [Architecture Overview](#)
- 2. [Development Environment Setup](#)
- 3. [Code Structure](#)
- 4. [Building and Testing](#)
- 5. [Adding New Translation Providers](#)
- 6. [File Format Support](#)
- 7. [Distributed System Development](#)
- 8. [API Development](#)
- 9. [Database Schema](#)
- 10. [Performance Optimization](#)
- 11. [Contributing Guidelines](#)

Architecture Overview

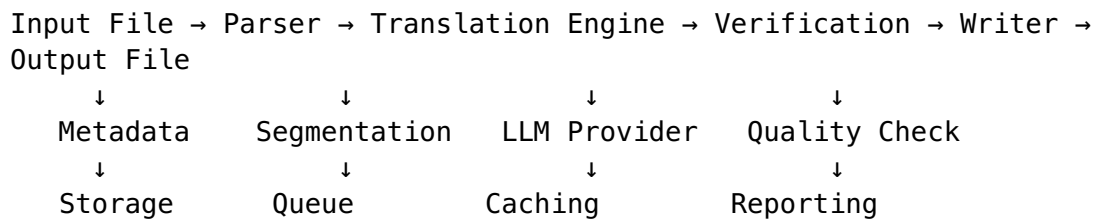
System Components

Web Interface/API			
Translation Engine			
LLM Providers	Dictionary Translation	Quality Verification	Distributed Processing
File Processing			
FB2 Parser	EPUB Parser	HTML Parser	TXT Parser
Storage Layer			
Database	Cache	File System	Queuing

Core Packages

- **pkg/translator**: Translation engine and LLM providers
- **pkg/ebook**: File format parsers and writers
- **pkg/distributed**: Distributed processing system
- **pkg/verification**: Quality verification system
- **pkg/api**: REST API implementation
- **pkg/websocket**: Real-time communication
- **internal/**: Internal utilities and configuration

Data Flow



Development Environment Setup

Prerequisites

- **Go**: Version 1.21 or higher
- **Git**: For version control
- **Docker**: For containerized development
- **Make**: For build automation
- **Node.js**: Version 18+ (for web interface development)

Setup Steps

1. Clone the Repository:

```
git clone https://github.com/digital-vasic/translator.git
cd translator
```

2. Install Dependencies:

```
go mod download
make deps
```

3. Set Environment Variables:

```
cp .env.example .env
# Edit .env with your API keys
```

4. Set Up Database (optional):

```
make db-setup
```

5. Run Tests:

```
make test
```

6. Start Development Server:

```
make dev
```

IDE Configuration

VS Code

Install these extensions:

- Go (golang.go)
- Docker (ms-azuretools.vscode-docker)
- REST Client (humao.rest-client)

Configure workspace settings:

```
{
  "go.useLanguageServer": true,
  "go.testFlags": ["-v"],
  "go.buildFlags": ["-v"],
  "go.lintOnSave": "package",
  "go.vetOnSave": "package"
}
```

GoLand

- Enable Go modules support
- Configure run configurations for tests
- Set up database integration
- Configure Docker integration

Code Structure

Project Layout

```
translator/
├── cmd/                # Command-line applications
│   ├── cli/           # CLI application
│   ├── server/        # Web server
│   └── worker/        # Distributed worker
├── pkg/               # Public packages
│   ├── translator/    # Translation engine
│   │   ├── llm/       # LLM providers
│   │   └── dictionary/ # Dictionary translation
│   ├── ebook/         # File format handling
│   ├── distributed/   # Distributed processing
│   ├── verification/  # Quality verification
│   ├── api/           # REST API
│   └── websocket/     # WebSocket handlers
```

└─ internal/	# Internal packages
└─ └─ config/	# Configuration management
└─ └─ cache/	# Caching layer
└─ └─ database/	# Database operations
└─ └─ utils/	# Utilities
└─ test/	# Test utilities and mocks
└─ docs/	# Documentation
└─ scripts/	# Build and deployment scripts
└─ web/	# Web interface assets
└─ configs/	# Configuration files

Package Dependencies

cmd/	→ pkg/translator, pkg/ebook, pkg/api
pkg/translator	→ internal/config, internal/cache
pkg/ebook	→ pkg/translator
pkg/distributed	→ pkg/translator, internal/database
pkg/api	→ pkg/translator, pkg/websocket
pkg/websocket	→ pkg/distributed
internal/	→ External dependencies only

Interface Contracts

Translation Provider Interface

```

type TranslationProvider interface {
    Translate(ctx context.Context, req *TranslationRequest)
        (*TranslationResponse, error)
    GetModels() ([]Model, error)
    ValidateConfig() error
    GetCapabilities() ProviderCapabilities
}

```

```

type TranslationRequest struct {
    Text      string
    SourceLang string
    TargetLang string
    Options   TranslationOptions
}

```

```

type TranslationResponse struct {
    TranslatedText string
    Confidence      float64
    Model           string
    TokensUsed      int
    Cost            float64
}

```

File Parser Interface

```
type Parser interface {
    Parse(ctx context.Context, data []byte) (*Ebook, error)
    Validate(data []byte) error
    SupportedFormats() []string
    GetMetadata(data []byte) (*Metadata, error)
}

type Writer interface {
    Write(ctx context.Context, ebook *Ebook) ([]byte, error)
    SupportedFormats() []string
}
```

Building and Testing

Build Commands

```
# Build all binaries
make build

# Build specific component
make build-cli
make build-server
make build-worker

# Build for different platforms
make build-all-platforms

# Build Docker images
make docker-build
```

Testing

```
# Run all tests
make test

# Run specific test suite
make test-unit
make test-integration
make test-e2e

# Run with coverage
make test-coverage

# Run benchmarks
make test-bench

# Race condition tests
make test-race
```

Code Quality

Lint code

`make lint`

Format code

`make fmt`

Security audit

`make security-audit`

Dependency check

`make deps-check`

Development Workflow

1. Create Feature Branch:

`git checkout -b feature/new-provider`

2. Make Changes with tests:

Write code

`go test ./pkg/translator/llm/...`

3. Run Full Test Suite:

`make test-all`

4. Commit Changes:

`git add .`

`git commit -m "feat: add new translation provider"`

5. Push and Create PR:

`git push origin feature/new-provider`

Adding New Translation Providers

Step 1: Create Provider Implementation

Create `pkg/translator/llm/newprovider/provider.go`:

package newprovider

import (

 "context"

 "fmt"

 "github.com/digital-vasic/translator/pkg/translator"

)

```

type Provider struct {
    config *Config
    client *Client
}

type Config struct {
    APIKey      string `yaml:"api_key"`
    BaseURL     string `yaml:"base_url"`
    Model       string `yaml:"model"`
    Temperature float64 `yaml:"temperature"`
    MaxTokens   int    `yaml:"max_tokens"`
}

type Client struct {
    apiKey string
    baseURL string
}

func NewProvider(config *Config) *Provider {
    return &Provider{
        config: config,
        client: &Client{
            apiKey: config.APIKey,
            baseURL: config.BaseURL,
        },
    }
}

func (p *Provider) Translate(ctx context.Context, req
    *translator.TranslationRequest)
    (*translator.TranslationResponse, error) {
    // Implementation here
    return &translator.TranslationResponse{
        TranslatedText: "Translated text",
        Confidence:      0.95,
        Model:          p.config.Model,
        TokensUsed:      100,
        Cost:            0.001,
    }, nil
}

func (p *Provider) GetModels() ([]translator.Model, error) {
    return []translator.Model{
        {
            Name:          "model-name",
            DisplayName:  "Model Display Name",
            MaxTokens:     4096,
            CostPer1K:     0.002,
        },
    }, nil
}

func (p *Provider) ValidateConfig() error {

```

```

    if p.config.APIKey == "" {
        return fmt.Errorf("API key is required")
    }
    return nil
}

func (p *Provider) GetCapabilities()
    translator.ProviderCapabilities {
    return translator.ProviderCapabilities{
        SupportsStreaming: true,
        SupportsBatch:     true,
        SupportsLanguages: []string{"en", "ru", "sr"},
        MaxInputTokens:    4096,
        MaxOutputTokens:   4096,
    }
}

```

Step 2: Create Tests

Create pkg/translator/llm/newprovider/provider_test.go:

```

package newprovider

import (
    "context"
    "testing"
    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestProvider_Translate(t *testing.T) {
    config := &Config{
        APIKey: "test-key",
        Model:  "test-model",
    }

    provider := NewProvider(config)

    req := &translator.TranslationRequest{
        Text:      "Hello, world!",
        SourceLang: "en",
        TargetLang: "sr",
    }

    resp, err := provider.Translate(context.Background(), req)
    require.NoError(t, err)
    assert.NotNil(t, resp)
    assert.NotEmpty(t, resp.TranslatedText)
    assert.Greater(t, resp.Confidence, 0.0)
}

func TestProvider_ValidateConfig(t *testing.T) {

```

```

tests := []struct {
    name      string
    config    *Config
    wantErr   bool
}{
    {
        name:      "valid config",
        config:    &Config{APIKey: "valid-key"},
        wantErr:   false,
    },
    {
        name:      "missing API key",
        config:    &Config{},
        wantErr:   true,
    },
}

for _, tt := range tests {
    t.Run(tt.name, func(t *testing.T) {
        provider := NewProvider(tt.config)
        err := provider.ValidateConfig()
        if tt.wantErr {
            assert.Error(t, err)
        } else {
            assert.NoError(t, err)
        }
    })
}

```

Step 3: Register Provider

Update pkg/translator/llm/registry.go:

```

import (
    // existing imports
    "github.com/digital-vasic/translator/pkg/translator/llm/
        newprovider"
)

func RegisterAll() {
    // existing providers
    providers["newprovider"] = func(config
        map[string]interface{}) (translator.Provider, error) {
        cfg := &newprovider.Config{}
        if err := mapstructure.Decode(config, cfg); err != nil {
            return nil, err
        }
        return newprovider.NewProvider(cfg), nil
    }
}

```

Step 4: Update Configuration

Add to configuration schema in `internal/config/types.go`:

```
type TranslationConfig struct {
    DefaultProvider string `yaml:"default_provider"`
    Providers       map[string]ProviderConfig `yaml:"providers"`
}

type ProviderConfig struct {
    Type      string `yaml:"type"`
    Settings map[string]interface{} `yaml:"settings"`
}
```

File Format Support

Adding New Parser

1. Implement Parser Interface:

```
type NewFormatParser struct {
    config *ParserConfig
}

func (p *NewFormatParser) Parse(ctx context.Context, data []byte)
    (*Ebook, error) {
    // Parse implementation
}

func (p *NewFormatParser) Validate(data []byte) error {
    // Validation logic
}

func (p *NewFormatParser) SupportedFormats() []string {
    return []string{"newformat"}
}

func (p *NewFormatParser) GetMetadata(data []byte) (*Metadata,
    error) {
    // Metadata extraction
}
```

1. Register Parser:

```
func RegisterParser(format string, parser Parser) {
    parsers[format] = parser
}
```

```
// In init()
RegisterParser("newformat", &NewFormatParser{})
```

1. Add Tests:

```
func TestNewFormatParser_Parse(t *testing.T) {
    parser := &NewFormatParser{}

    // Test with valid data
    data := []byte("...valid newformat data...")
    ebook, err := parser.Parse(context.Background(), data)
    require.NoError(t, err)
    assert.NotNil(t, ebook)

    // Test with invalid data
    invalidData := []byte("invalid data")
    _, err = parser.Parse(context.Background(), invalidData)
    assert.Error(t, err)
}
```

Adding New Writer

Similar to parser, implement the Writer interface:

```
type NewFormatWriter struct {
    config *WriterConfig
}

func (w *NewFormatWriter) Write(ctx context.Context, ebook
    *Ebook) ([]byte, error) {
    // Write implementation
}

func (w *NewFormatWriter) SupportedFormats() []string {
    return []string{"newformat"}
}
```

Distributed System Development

Node Registration

```
type Node struct {
    ID          string    `json:"id"`
    Address     string    `json:"address"`
    Status      string    `json:"status"`
    Load       float64   `json:"load"`
    LastSeen    time.Time `json:"last_seen"`
}

func (c *Coordinator) RegisterNode(ctx context.Context, node
    *Node) error {
```

```

    // Register node with the coordinator
    return c.registry.Register(node)
}

```

Task Distribution

```

type Task struct {
    ID          string          `json:"id"`
    Type        string          `json:"type"`
    Payload     interface{}      `json:"payload"`
    Options     map[string]interface{} `json:"options"`
    Status      string          `json:"status"`
}

func (c *Coordinator) DistributeTask(ctx context.Context, task
    *Task) error {
    // Select appropriate node based on load and capabilities
    node := c.selectNode(task)

    // Send task to selected node
    return c.sendTask(node, task)
}

```

Load Balancing

```

type LoadBalancer struct {
    strategy string
    nodes    []*Node
}

func (lb *LoadBalancer) SelectNode(task *Task) *Node {
    switch lb.strategy {
    case "round_robin":
        return lb.roundRobinSelect()
    case "least_loaded":
        return lb.leastLoadedSelect()
    case "capability_based":
        return lb.capabilityBasedSelect(task)
    default:
        return lb.roundRobinSelect()
    }
}

```

API Development

Adding New Endpoint

1. Define Handler:

```
// pkg/api/handlers.go
func (h *Handler) NewEndpoint(w http.ResponseWriter, r
    *http.Request) {
    var req NewEndpointRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        http.Error(w, "Invalid request", http.StatusBadRequest)
        return
    }

    // Process request
    result, err := h.service.ProcessNewEndpoint(req)
    if err != nil {
        http.Error(w, err.Error(), http.StatusInternalServerError)
        return
    }

    w.Header().Set("Content-Type", "application/json")
    json.NewEncoder(w).Encode(result)
}
```

1. Register Route:

```
// pkg/api/router.go
func (r *Router) setupRoutes() {
    r.HandleFunc("/api/v1/new-endpoint",
        h.NewEndpoint).Methods("POST")
}
```

1. Add Documentation:

Update OpenAPI specification in docs/api/openapi.yaml:

```
paths:
  /api/v1/new-endpoint:
    post:
      summary: New endpoint description
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/NewEndpointRequest'
      responses:
        '200':
          description: Successful response
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/NewEndpointResponse'
```

Middleware

```
func AuthMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        token := r.Header.Get("Authorization")
        if !isValidToken(token) {
            http.Error(w, "Unauthorized", http.StatusUnauthorized)
            return
        }

        next.ServeHTTP(w, r)
    })
}

func LoggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r
        *http.Request) {
        start := time.Now()

        // Log request
        log.Printf("%s %s %s", r.Method, r.URL.Path, r.RemoteAddr)

        // Process request
        next.ServeHTTP(w, r)

        // Log response time
        log.Printf("Request processed in %v", time.Since(start))
    })
}
```

Database Schema

Migration Files

Create migration in migrations/001_initial_schema.up.sql:

```
CREATE TABLE translations (
    id SERIAL PRIMARY KEY,
    source_text TEXT NOT NULL,
    translated_text TEXT NOT NULL,
    source_lang VARCHAR(10) NOT NULL,
    target_lang VARCHAR(10) NOT NULL,
    provider VARCHAR(50) NOT NULL,
    model VARCHAR(100),
    confidence DECIMAL(3,2),
    tokens_used INTEGER,
    cost DECIMAL(10,6),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```

CREATE INDEX idx_translations_lang_pair ON
    translations(source_lang, target_lang);
CREATE INDEX idx_translations_provider ON translations(provider);
CREATE INDEX idx_translations_created_at ON
    translations(created_at);

```

Models

```

type Translation struct {
    ID            int           `json:"id" db:"id"`
    SourceText    string        `json:"source_text" db:"source_text"`
    TranslatedText string      `json:"translated_text" db:"translated_text"`
    SourceLang    string        `json:"source_lang" db:"source_lang"`
    TargetLang    string        `json:"target_lang" db:"target_lang"`
    Provider      string        `json:"provider" db:"provider"`
    Model         *string      `json:"model" db:"model"`
    Confidence    *float64     `json:"confidence" db:"confidence"`
    TokensUsed    *int         `json:"tokens_used" db:"tokens_used"`
    Cost          *float64     `json:"cost" db:"cost"`
    CreatedAt     time.Time    `json:"created_at" db:"created_at"`
    UpdatedAt     time.Time    `json:"updated_at" db:"updated_at"`
}

```

Repository Pattern

```

type TranslationRepository interface {
    Create(ctx context.Context, translation *Translation) error
    GetByID(ctx context.Context, id int) (*Translation, error)
    GetByLangPair(ctx context.Context, sourceLang, targetLang
        string, limit, offset int) ([]*Translation, error)
    Update(ctx context.Context, translation *Translation) error
    Delete(ctx context.Context, id int) error
}

type translationRepository struct {
    db *sqlx.DB
}

func NewTranslationRepository(db *sqlx.DB) TranslationRepository {
    return &translationRepository{db: db}
}

func (r *translationRepository) Create(ctx context.Context,
    translation *Translation) error {
    query := `
        INSERT INTO translations (source_text, translated_text,
            source_lang, target_lang, provider, model, confidence,
            tokens_used, cost)
        VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9)
        RETURNING id, created_at, updated_at
    `
}

```

```

        return r.db.QueryRowContext(ctx, query,
            translation.SourceText,
            translation.TranslatedText,
            translation.SourceLang,
            translation.TargetLang,
            translation.Provider,
            translation.Model,
            translation.Confidence,
            translation.TokensUsed,
            translation.Cost,
        ).Scan(&translation.ID, &translation.CreatedAt,
            &translation.UpdatedAt)
    }
}

```

Performance Optimization

Caching Strategy

```

type Cache struct {
    redis *redis.Client
    local *lru.Cache
    ttl    time.Duration
}

func (c *Cache) Get(key string) (interface{}, error) {
    // Try local cache first
    if value, ok := c.local.Get(key); ok {
        return value, nil
    }

    // Try Redis cache
    value, err := c.redis.Get(context.Background(), key).Result()
    if err == nil {
        // Store in local cache
        c.local.Add(key, value)
        return value, nil
    }

    return nil, err
}

func (c *Cache) Set(key string, value interface{}, ttl
    time.Duration) error {
    // Set in Redis
    if err := c.redis.Set(context.Background(), key, value,
        ttl).Err(); err != nil {
        return err
    }
}

```

```

    // Set in local cache
    c.local.Add(key, value)
    return nil
}

```

Database Optimization

```

func (r *translationRepository) GetByLangPairOptimized(ctx
    context.Context, sourceLang, targetLang string, limit,
    offset int) ([]*Translation, error) {
    // Use prepared statements
    query := `
        SELECT id, source_text, translated_text, source_lang,
            target_lang,
                provider, model, confidence, tokens_used, cost,
            created_at, updated_at
        FROM translations
        WHERE source_lang = $1 AND target_lang = $2
        ORDER BY created_at DESC
        LIMIT $3 OFFSET $4
    `

    stmt, err := r.db.PrepareContext(ctx, query)
    if err != nil {
        return nil, err
    }
    defer stmt.Close()

    var translations []*Translation
    err = stmt.SelectContext(ctx, &translations, sourceLang,
        targetLang, limit, offset)
    return translations, err
}

```

Memory Management

```

type MemoryPool struct {
    buffers sync.Pool
}

func NewMemoryPool() *MemoryPool {
    return &MemoryPool{
        buffers: sync.Pool{
            New: func() interface{} {
                return make([]byte, 0, 4096)
            },
        },
    }
}

func (p *MemoryPool) GetBuffer() []byte {
    return p.buffers.Get().([]byte)[:0]
}

```

```

}

func (p *MemoryPool) PutBuffer(buf []byte) {
    if cap(buf) >= 4096 { // Only reuse reasonable sized buffers
        p.buffers.Put(buf)
    }
}

func (c *Client) ProcessLargeText(text string) error {
    buf := c.pool.GetBuffer()
    defer c.pool.PutBuffer(buf)

    // Process text in chunks
    chunks := chunkText(text, 4096)
    for _, chunk := range chunks {
        buf = append(buf, chunk...)
        // Process buffer
        processChunk(buf)
        buf = buf[:0] // Reset buffer
    }

    return nil
}

```

Contributing Guidelines

Code Standards

1. **Go Conventions:** Follow [Effective Go](#)
2. **Naming:** Use descriptive names, avoid abbreviations
3. **Comments:** Document public functions and complex logic
4. **Error Handling:** Always handle errors, use descriptive error messages
5. **Testing:** Aim for 80%+ coverage, write tests for new features

Pull Request Process

1. Create Feature Branch:

```
git checkout -b feature/your-feature-name
```

2. Make Changes:

- Write code following the standards
- Add comprehensive tests
- Update documentation

3. Run Tests:

```
make test-all
```

4. Submit PR:

- Provide clear description
- Link to relevant issues
- Include screenshots for UI changes

Code Review Checklist

☐

Code follows style guidelines

☐

Tests cover edge cases

☐

Documentation is updated

☐

No hardcoded values

☐

Error handling is comprehensive

☐

Security considerations addressed

☐

Performance impact considered

Release Process

1. Update Version:

```
# Update version in go.mod
# Update CHANGELOG.md
# Create Git tag
git tag v1.2.3
```

2. Build Release:

```
make release
```

3. Deploy:

```
make deploy
```

Security Considerations

1. **API Keys:** Never commit API keys, use environment variables
2. **Input Validation:** Always validate user input
3. **SQL Injection:** Use parameterized queries
4. **XSS Prevention:** Sanitize output for web interface
5. **Rate Limiting:** Implement rate limiting for API endpoints

Performance Benchmarks

Run benchmarks regularly:

`make bench`

Target performance metrics:

- **Translation API:** <2 seconds average response time
 - **File Processing:** <5 seconds per 100KB
 - **Concurrent Users:** Support 1000+ concurrent users
 - **Memory Usage:** <512MB for typical workload
-

Resources

- [Go Documentation](#)
- [Effective Go](#)
- [Go Code Review Comments](#)
- [API Style Guide](#)
- [Database Design Best Practices](#)